TU Dortmund University
Department of Computer Science · Chair of Enterprise Computing

Case 1: Artificial Intelligence

Machine Learning Applications

Advanced Enterprise Computing: Management of AI
WS 26/27

Lecturer	Prof. Dr. Christian Janiesch
Exercise instructor	Seyyid Ahmed Ciftci

Name	Matriculation number	Email
Mohamed Gomaa	283950	mohamed.gomaa@tu-dortmund.de
Ahmed Farag	284711	ahmed.farag@tu-dortmund.de
Ahmed Abdelrazek	284304	ahmed.abdelrazek@tu-dortmund.de

Repository: github.com/Advanced-Enterprise-Computing/case1-artificial-intelligence

Contents

1 TensorFlow Playground 3

1.1 A1 – Linear model on the ring 3

1.2 A2 – Hidden layer and model capacity 4

1.3 B1 – Learning rate 0.003 vs. 1.0 6

1.4 B2 – L2 regularization under 30 % noise 7

1.5 Summary and link to Part 2 10

2 Image Classification (Cat vs. Dog) 11

2.1 Setup (model, data, preprocessing) 11

2.2 Experiment 1 – Train/test split sizes 11

2.3 Experiment 2 – Optimizers 12

2.4 Metrics 12

2.5 Visualizations (table and graphs) 13

2.6 Discussion – why the results differ 15

References 17

A Appendix: Jupyter notebook 18

List of figures

Figure A1 No hidden layer: the boundary is a straight line 4

Figure A2 Best decision boundary: 1 hidden layer with 6 neurons, Tanh 5

Figure A2b Output with 6, 4, 3 and 2 hidden neurons 6

Figure B1 Output with learning rates 0.003, 0.03 and 1.0 7

Figure B2a Output without and with L2 regularization, at 0 % and 30 % noise 9

Figure B2b Setup and result: L2 regularization at 30 % noise 10

Figure 1 Accuracy and loss after every epoch 14

Figure 2 Confusion matrices 14

Figure 3 Test accuracy with 95 % confidence interval 15

Figure 4 ROC curves 15

1 TensorFlow Playground

Setup. All runs use the TensorFlow Playground (Smilkov & Carter, n.d.) with:

- Problem type = Classification and the Circle dataset: 500 points, a blue disk in the center surrounded by an orange ring.
- A 50 % ratio of training to test data (250 points each), with Show test data on. Test points have a black outline, training points a white one.
- Only the features x1 and x2, batch size 10 and the Tanh activation.
- Unless stated otherwise, a learning rate of 0.03 and no regularization.

What the numbers mean. The Playground reports the training loss and the test loss, not the accuracy. The loss is the average of $\frac{1}{2} \cdot (\text{output} - \text{label})^2$, where the label is -1 (orange) or $+1$ (blue) and the output lies between -1 and $+1$. A loss of 0 means every point is classified correctly and with full confidence. A loss of 0.5 is what a model gets if its output is 0 everywhere, i.e. if it cannot decide.

Making runs comparable. The Playground stores all settings, including the random seed, in its URL. We fixed the seed (0.42000) so that every run starts with the same data and the same initial weights. We used the step button to stop every run at exactly the stated epoch (200, and 1,000 for the longer runs), so the numbers in the text match the screenshots. We repeated the key comparisons with two further seeds (0.11111 and 0.77777). We also set the noise through the URL, because moving the noise slider by hand creates a new random dataset.

1.1 A1 – Linear model on the ring

Setting: 0 hidden layers, learning rate 0.03, no regularization (Figure A1).

What we see. The decision boundary is a straight line that runs diagonally through the middle of the plot. The background is pale blue on one side and pale orange on the other, so the model is unsure everywhere. Both sides contain blue and orange points: roughly 40 % of the points lie on the wrong side. The lines from x1 and x2 to the output are almost invisible, which means the weights are close to 0. The loss drops from 0.53 to 0.50 within the first ten epochs and then stays there (training loss 0.501 and test loss 0.499 after 200 epochs). Training longer does not help.

Why the ring cannot be separated. Without a hidden layer, the network computes output = $\tanh(w1 \cdot x1 + w2 \cdot x2 + b)$, and the points where the output is 0 form a straight line: the model is linear. The two classes are not linearly separable, because the blue points form a disk and the orange points a ring around it. Every straight line cuts through the ring and leaves blue and orange points on both sides. No line can enclose the disk.

The best the model can do is to hedge. It keeps its weights small, so its output stays close to 0 everywhere (the pale colors and the faint lines). This gives the loss of 0.5 of an undecided model. Both losses stay flat and high, which the Case 1 slides describe as underfitting (Ciftci, 2026): the model has too little capacity for the shape of the data. More training does not fix this; a more flexible model does.

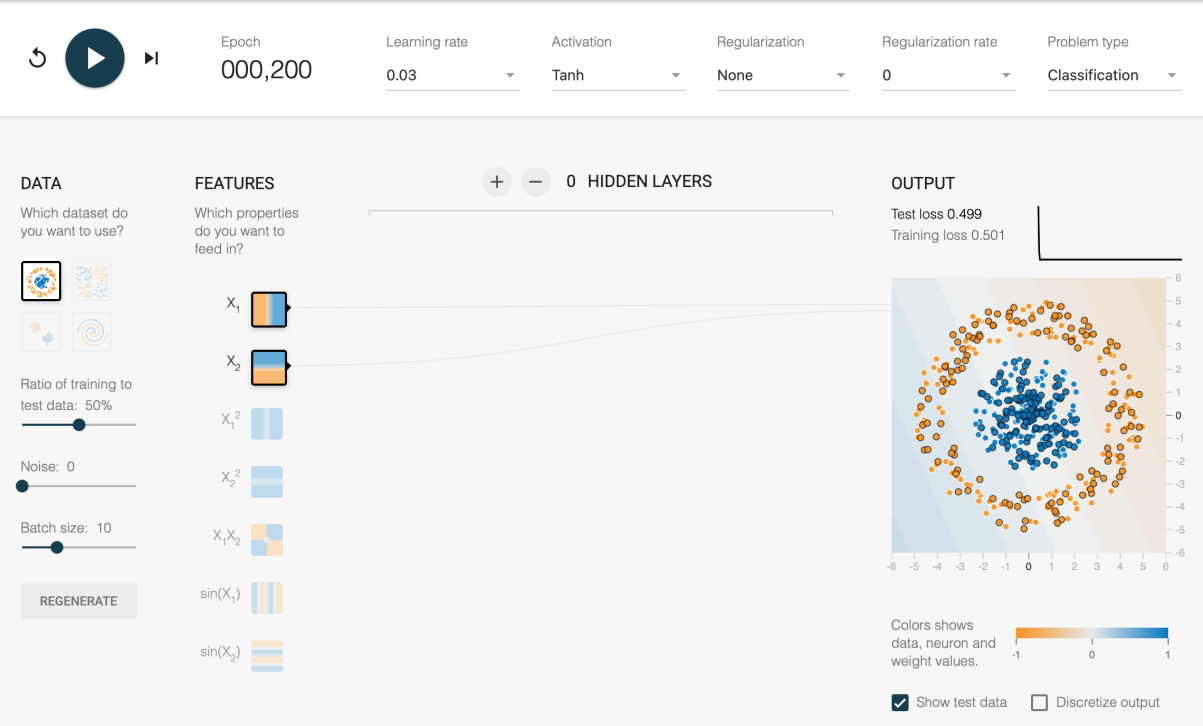


Figure A1 – No hidden layer, learning rate 0.03, after 200 epochs. The pale, almost uniform background shows a straight boundary with low confidence on both sides. Both losses stay at 0.50, as the flat loss curve at the top right shows.

1.2 A2 – Hidden layer and model capacity

Setting: 1 hidden layer with 6 neurons, Tanh, learning rate 0.03, no regularization.

How the boundary changes. The boundary is no longer a line. It is a closed, roughly round curve that encloses the blue disk and runs through the empty gap between the disk and the ring (Figure A2). The test loss falls quickly: 0.31 after 10 epochs, 0.06 after 50, 0.024 after 100 and 0.013 after 200 epochs (training loss 0.012). Every training and test point lies on the correct side.

Fewer neurons. Each run below lasted 200 epochs with the same seed (Figure A2b):

Neurons in the hidden layer	Shape of the boundary	Training loss	Test loss
6	closed, almost round polygon	0.012	0.013
4	closed, rounded four-sided shape	0.018	0.018
3 (extra test)	closed, rounded triangle	0.019	0.016
2	open diagonal band	0.264	0.265

With 4 neurons the ring is still separated cleanly. With 2 neurons it is not. The boundary becomes a band across the whole plot, which contains the blue disk but also two parts of the orange ring, and about one in five points ends up on the wrong side. Of 6, 4 and 2, the smallest count that still separates the ring well is therefore 4. An extra test showed that 3 neurons also work. The result held with two further seeds: 3 neurons reached a test loss of 0.022 and 0.025, while 2 neurons failed again with a test loss of 0.32.

What this reveals about capacity. Every hidden neuron receives only x1 and x2, so on its own it can only draw one soft straight edge. The Playground shows this directly: the small picture of each hidden neuron in Figure A2 is split by a single line. The output neuron combines these edges:

- To enclose a region in a plane, at least three edges are needed (a triangle).
- With two edges, only a band or a wedge is possible, and both are open to the outside.
- More neurons give more edges and therefore a rounder curve: 3 neurons give a rounded triangle, 4 a rounded four-sided shape, 6 an almost round polygon. Beyond the minimum, they make the boundary smoother rather than the separation better.

Informally, capacity is a model's ability to fit a wide variety of functions (Goodfellow et al., 2016, Section 5.2). Here it is the range of shapes the boundary can take: no hidden neuron only lines (A1), 2 neurons only bands, 3 or more neurons closed shapes. The model needs enough capacity for the shape of the data, here a closed curve. In theory, a single hidden layer with enough neurons can approximate any continuous function (Hornik et al., 1989); the experiment shows how many are enough for the ring: three.

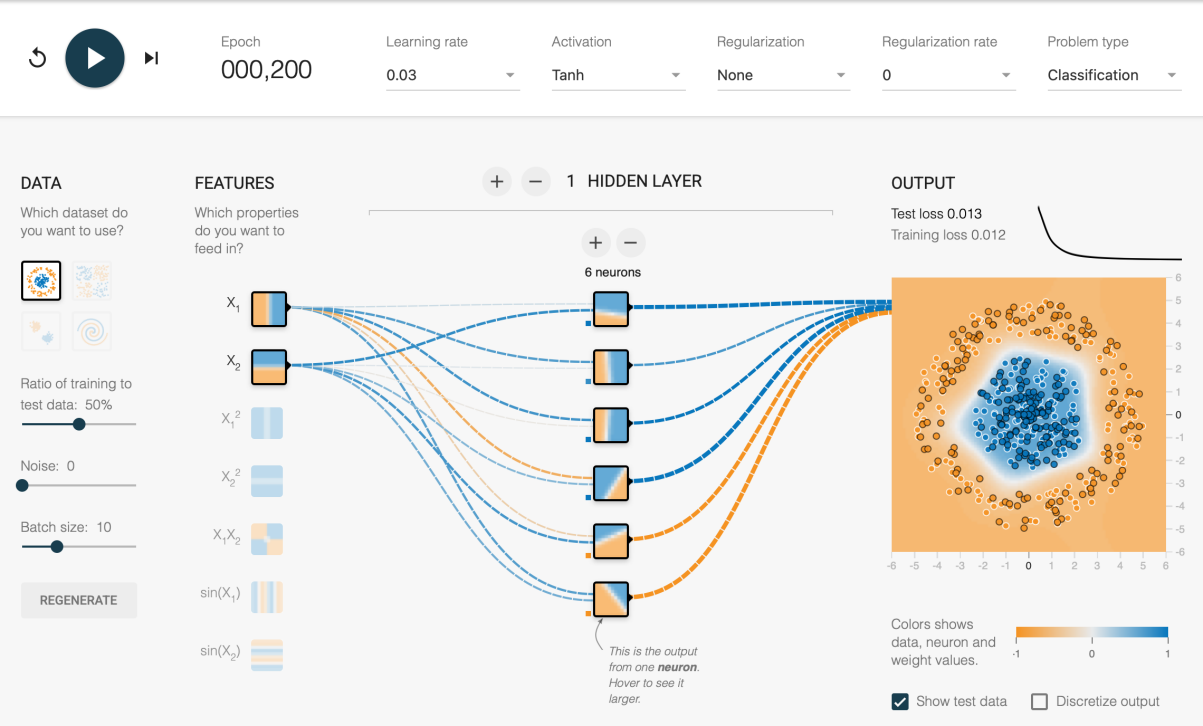


Figure A2 – Best decision boundary (1 hidden layer, 6 neurons, Tanh, learning rate 0.03, after 200 epochs). These visual aspects show its quality:

- **Clean separation:** the boundary is a closed curve with every blue point inside and every orange point outside, for the training points and for the test points (black outline).
- **Placement:** it runs through the middle of the empty gap between the two classes, so small shifts of a point would not change its class.
- **Smooth shape:** it is smooth and roughly round, like the data itself, without bumps around single points.

- **Confidence:** the colors are strong on both sides and fade only in a narrow band inside the gap, so the network is confident wherever there are points.
- **Generalization and convergence:** the test loss (0.013) is as low as the training loss (0.012), and the loss curve at the top right has flattened.

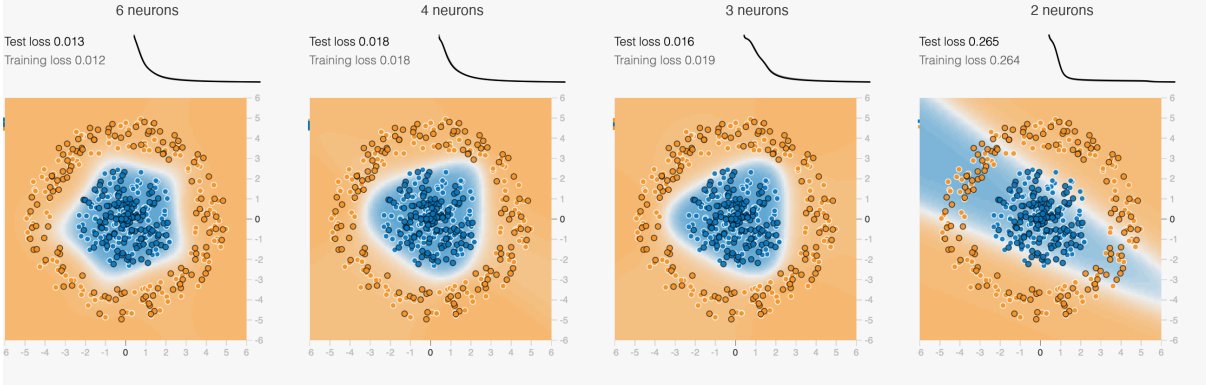


Figure A2b – Output after 200 epochs with 6, 4, 3 and 2 neurons in the hidden layer. With 3 or more neurons the boundary is closed. With 2 neurons it is an open band that also covers parts of the orange ring.

1.3 B1 – Learning rate 0.003 vs. 1.0

Setting: the architecture from A2 (1 hidden layer, 6 neurons, Tanh), no noise, no regularization. The default 0.03 is shown for reference.

Learning rate	Test loss after 10 epochs	after 50	after 200	Course of the loss
0.003	0.480	0.405	0.174	smooth, falls steadily
0.03 (A2)	0.305	0.057	0.013	smooth
1.0	0.014	0.003	0.003	jumps up and down at the start

Speed. With 0.003 the steps are ten times smaller than with 0.03, and training is about ten times slower. After 200 epochs the test loss is still 0.17 and the boundary is only a pale outline, with about 5 % of the points still on the wrong side (Figure B1, left). It takes about 1,000 epochs to reach the test loss that 0.03 reaches after 100 epochs (0.024). With 1.0 the test loss fell below 0.03 within 20 epochs in two of our three runs.

Stability. With 0.003 the loss falls steadily: it did not jump up a single time in 1,000 epochs, and the boundary grows gradually. With 1.0 the steps are so large that the weights overshoot. The loss curve is jagged, and the boundary visibly jumps from epoch to epoch:

- In two of our three runs (different seeds), it calmed down after 20 to 40 epochs.
- In the third run it never settled. Between epochs 100 and 200 the test loss still jumped between 0.06 and 0.20. After 1,000 epochs the training loss was 0.024 but the test loss 0.088.
- This run was not even reproducible. With the same settings and seed, a second browser showed a test loss of 0.14 instead of 0.090 after 200 epochs, while every stable setting gave identical numbers in both browsers. With such large steps, tiny rounding differences grow into visibly different results.

Final boundary quality.

- **0.003:** given enough epochs, it ends with a smooth, round boundary, like 0.03.
- **1.0:** it ends with a very sharp boundary: hard color edges, a narrow transition and a more angular shape. The large steps drive the weights to large values and push the Tanh neurons into saturation.
  - When the run settles, the loss is very low (0.003), but the shape depends on luck.
  - In the run that did not settle, the region is a hard-edged triangle whose lower tip reaches into the orange ring (Figure B1, right).

Conclusion. A small learning rate is safe but slow, and a very large one is fast but unreliable, as the Case 1 slides predict (Ciftci, 2026). Here the default of 0.03 is the best compromise: fast and stable. This sensitivity is why Goodfellow et al. (2016, Chapter 11) describe the learning rate as perhaps the most important hyperparameter to tune.

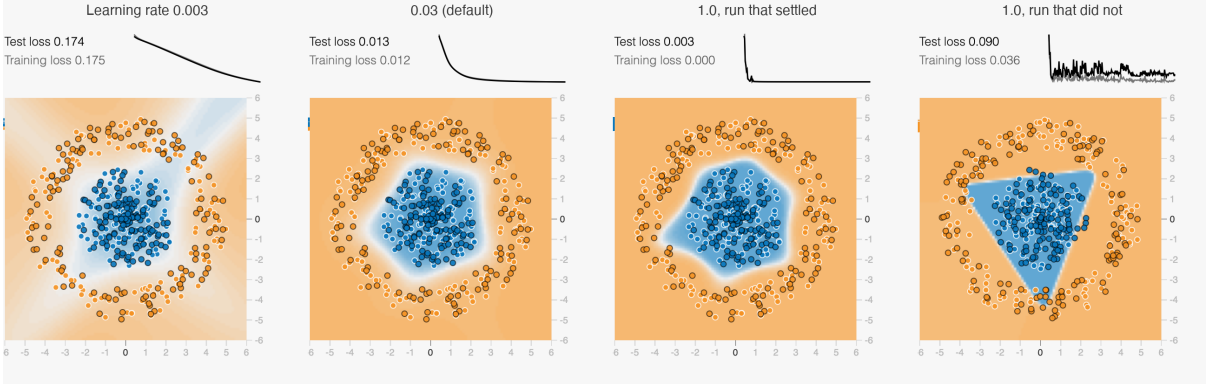


Figure B1 – Output after 200 epochs with learning rates 0.003, 0.03 and 1.0 (two runs with different seeds). The small chart above each plot shows the test loss (black) and the training loss (gray) over time. With 0.003 the boundary is still pale and the loss is still falling. With 1.0 the boundary has hard edges, and in the run that did not settle the loss curve is jagged.

1.4 B2 – L2 regularization under 30 % noise

Setting: the architecture from A2, learning rate 0.03, L2 regularization with rate 0.03, noise first 0 %, then 30 %. For comparison we ran exactly the same settings without regularization. The same seed means the same data and the same starting weights.

Noise	Regularization	Training loss	Test loss	Epochs
0 %	none	0.012	0.013	200
0 %	L2, 0.03	0.051	0.050	200
30 %	none	0.138 → 0.127	0.178 → 0.194	200 → 1,000
30 %	L2, 0.03	0.174 → 0.161	0.183 → 0.179	200 → 1,000

Is the boundary smoother with L2? Yes. Even without noise, the boundary with L2 is rounder and softer: the colors fade over a wider band (Figure B2a, top). L2 keeps the weights small: the lines in the network diagram, whose thickness shows the size of each weight, are visibly thinner. Small weights mean gentle Tanh curves instead of sharp steps. The price, without noise, is a higher loss (test loss 0.050 instead of 0.013): the outputs are less extreme, although every point is still classified correctly.

How test performance changes with 30 % noise. With noise, some points carry the color of the other class for their position. No boundary can classify everything correctly, and both losses rise to about 0.18. Both settings classify about 88 % of the test points correctly. After 200 epochs they are almost equal (test loss 0.178 without and 0.183 with L2). The difference appears with longer training:

- **Without L2:** the training loss keeps falling (0.138 → 0.127 at epoch 1,000), while the test loss rises (0.178 → 0.194). The weights keep growing, and the boundary loses its round shape: it gets a dent and a bulge where it bends toward individual noisy points (Figure B2a, bottom left). The Case 1 slides describe exactly this pattern: a validation loss that falls and then rises means overfitting (Ciftci, 2026).
- **With L2:** both losses stay flat (test loss 0.183 → 0.179), and the gap between training and test loss stays small (0.018 instead of 0.067 after 1,000 epochs). The weights stay about half as large, and the boundary stays smooth, symmetric and roughly round (Figure B2a, bottom right). L2 has even switched one of the six hidden neurons off: in Figure B2b its picture is empty and its lines are almost invisible.

Which setting is more robust, and why? L2 on. Its result hardly depends on how long we train or on individual noisy points, and its training loss is an honest estimate of its test loss. We repeated the comparison with two further seeds:

- In all three runs, L2 kept the gap between training and test loss smaller (0.008–0.075, against 0.053–0.094 without L2 after 1,000 epochs).
- With L2, the test loss never rose with longer training. Without L2, it rose in two of the three runs.
- The gain in test loss itself was small: after 200 epochs, the model without L2 was even slightly ahead in all three runs (by 0.005–0.014). With only 6 neurons the model has little capacity to memorize noise, so here L2 mainly stops the slow drift toward overfitting. With a larger network we would expect a bigger benefit.

The reason is how L2 works. Also called weight decay, it adds the sum of the squared weights, multiplied by the rate, to the loss (Ciftci, 2026; Goodfellow et al., 2016, Section 7.1.1), so every weight is pulled toward 0 unless it clearly helps many points. Bending the boundary around a single noisy point would need large weights for a sharp local curve, and the penalty makes that expensive. The boundary therefore follows the overall shape of the data, a circle, and ignores single outliers.

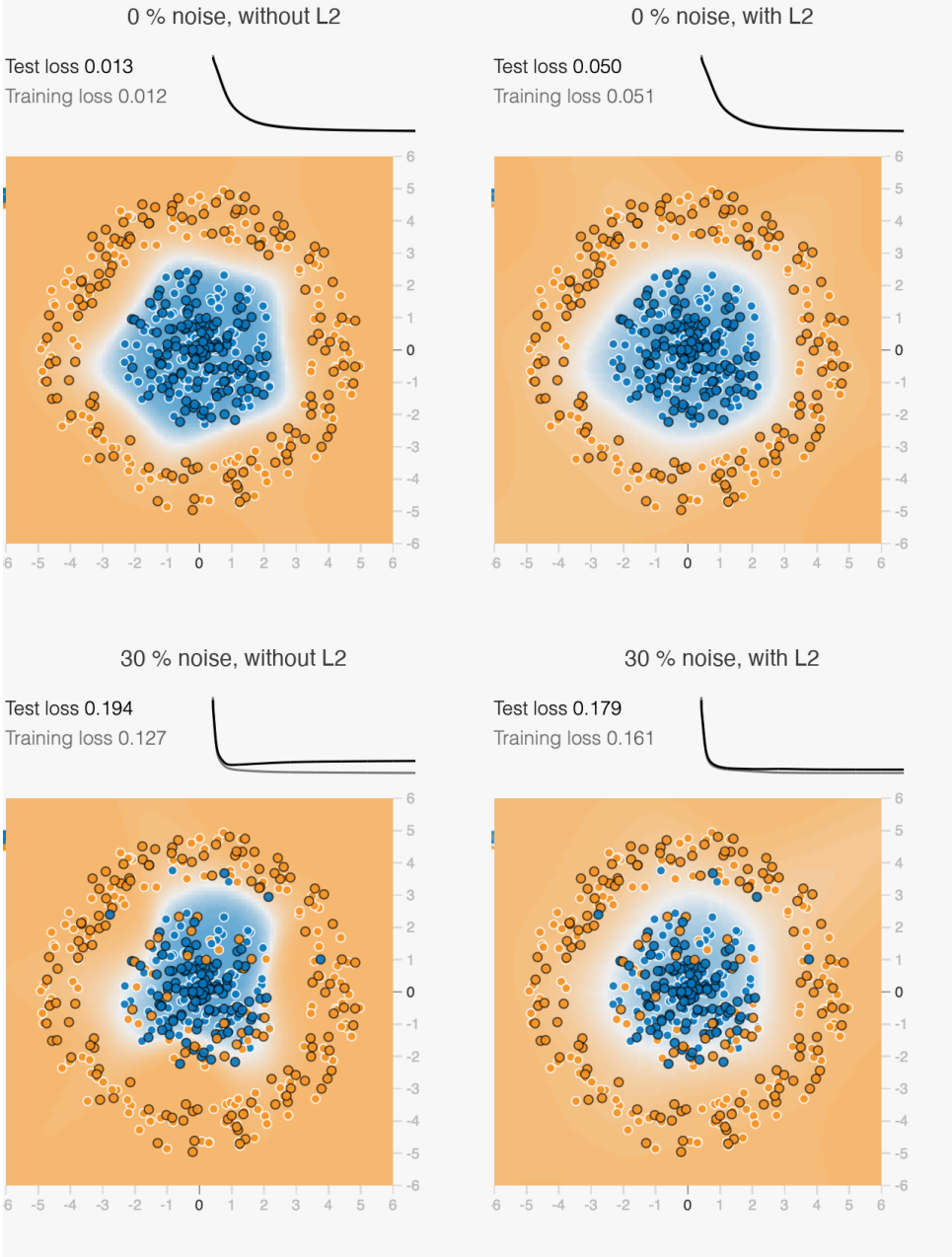


Figure B2a – Output without L2 (left) and with L2 regularization (right): at 0 % noise after 200 epochs (top) and at 30 % noise after 1,000 epochs (bottom). With L2 the boundary is rounder and its edges are softer. At 30 % noise without L2, the boundary has a dent and a bulge, and the loss chart shows the test loss (black) slowly rising above the falling training loss (gray). With L2 both curves stay flat and close together.

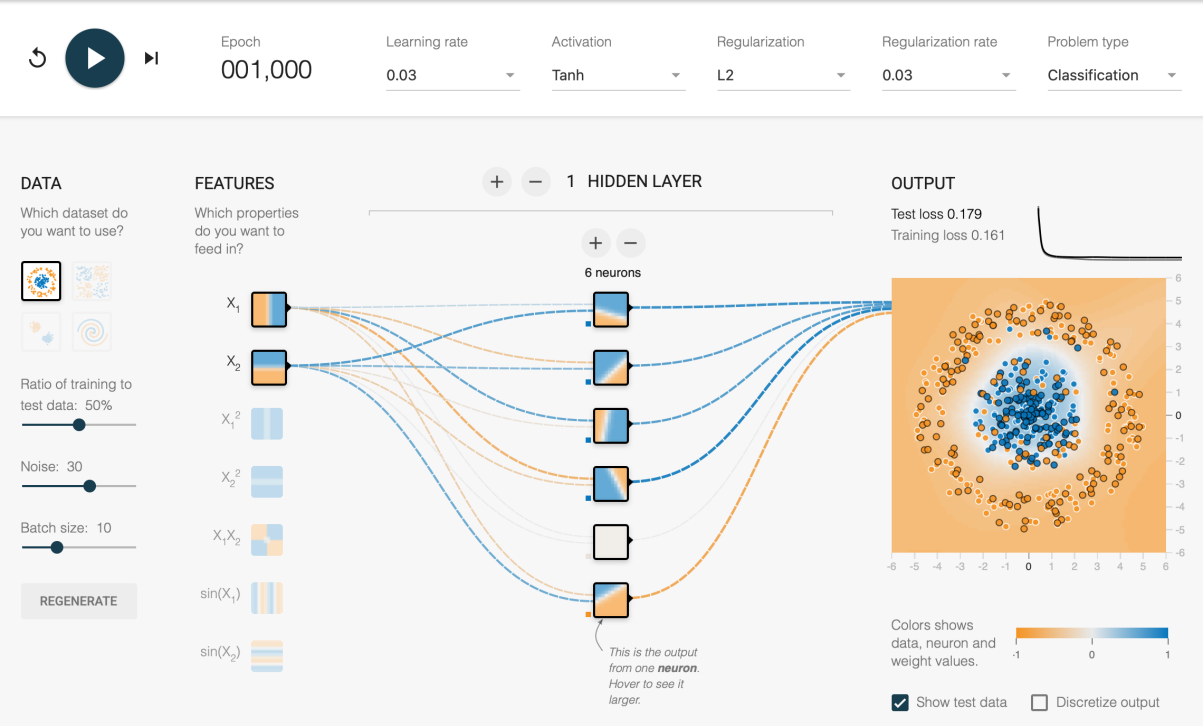


Figure B2b – Setup and result: L2 regularization with rate 0.03 at 30 % noise, after 1,000 epochs. Test loss 0.179, training loss 0.161. The fifth hidden neuron has been switched off by the regularization.

1.5 Summary and link to Part 2

- **Capacity:** a linear model cannot separate the ring (A1). A hidden layer with a nonlinear activation can, but it needs at least three neurons to enclose a region (A2).
- **Learning rate:** too small is slow but steady, too large is fast but unstable (B1).
- **L2 regularization:** smaller weights give a smoother boundary and, under noise, a stable test loss and a small gap between training and test loss (B2).

The hidden layer in A2 plays the same role as the frozen ResNet-18 in Part 2. It turns the raw input into features in which the classes become linearly separable, so the output neuron only has to draw a straight line in that feature space. In Part 2 the pretrained features are already good enough for one linear layer to reach about 98.5 % accuracy; in the Playground, the network has to learn its features itself.

2 Image Classification (Cat vs. Dog)

2.1 Setup (model, data, preprocessing)

Goal. Train a classifier that decides whether an image shows a cat or a dog, and measure how well it works on images it has not seen during training. In the terms of Mitchell (1997) used in the lecture (Janiesch, 2025), the task T is to label an image as cat or dog, the experience E is the set of labeled training images, and the performance measure P is the accuracy, complemented by the metrics below. This makes it a supervised, dichotomous classification problem (Janiesch et al., 2021).

Data. We use the provided subset of the Microsoft Cats vs Dogs dataset from Kaggle: 12,499 image files (6,203 cats, 6,296 dogs). One cat file is empty and is removed by the notebook's image check, which leaves 12,498 valid images. About 1 % of the files are BMP, GIF or PNG images despite their .jpg extension; Pillow reads them without problems. Both classes are almost equally large (49.6 % cats, 50.4 % dogs), so a trivial model that always answers "Dog" would only reach about 50 % accuracy. This is the baseline every real model has to beat.

Preprocessing. Every image is resized to 256 px, center-cropped to 224 × 224 px and normalized with the ImageNet mean and standard deviation. These are exactly the transforms the pretrained weights expect (`ResNet18_Weights.IMAGENET1K_V1.transforms()`).

Model (transfer learning). We start from a ResNet-18 (He et al., 2016) that was pretrained on ImageNet and freeze all of its layers. Only the last fully connected layer is replaced by a new layer with two outputs (Cat, Dog), and only this layer is trained. It has $512 \times 2 + 2 = 1{,}026$ parameters, so the network reuses the visual features it learned on ImageNet and only learns how to combine them for our two classes.

Training. Cross-entropy loss, learning rate 0.001, batch size 32, 3 epochs per run. Every run uses the same random seed (42), so it gets the same data split and the same starting weights, and only the factor we vary changes. The model is evaluated on the test split after every epoch. One run took about 5.5 minutes on an NVIDIA RTX 2060 GPU.

We ran 2 split sizes × 2 optimizers = 4 experiments. A first single-epoch run (80/20, Adam) already reached a test accuracy of 0.984.

2.2 Experiment 1 – Train/test split sizes

We compared two ways of splitting the 12,498 images:

Split	Training images	Test images
80/20	9,998	2,500
50/50	6,249	6,249

With 80/20 the models reach a test accuracy of about 98.5 %, with 50/50 about 97.9 %, for both optimizers (Table 1). With fewer training images, the new last layer sees fewer examples of each class and learns a slightly less accurate decision boundary. The confusion matrices (Figure 2) show where the difference comes from: cats are recognized equally well with both splits (98.4–98.8 %), but with 50/50 the models call 2.6–2.9 % of the dogs "cat", compared

with 1.4–1.8 % with 80/20. A plausible explanation is that dogs vary much more in appearance (breeds, sizes, coats) than cats, so they need more training examples.

The split size also changes how reliable the measurement is. The 50/50 test set is 2.5 times larger, so its accuracy has a smaller margin of error: ± 0.35 percentage points instead of about ± 0.47 (95 % confidence interval, Figure 3). Choosing a split is therefore a trade-off: more training data gives a better model, more test data gives a more trustworthy estimate of how good the model is.

The training time was almost the same for both splits (5.3–5.6 minutes). Because the backbone is frozen, most of the time goes into passing images through ResNet-18, and every epoch processes all 12,498 images once (training plus test), however they are split.

2.3 Experiment 2 – Optimizers

We compared Adam (Kingma & Ba, 2015) with stochastic gradient descent (SGD) with momentum 0.9, both with a learning rate of 0.001. Plain SGD would converge very slowly at this learning rate, so we added momentum.

Within each split, both optimizers end at practically the same result: 0.9852 vs. 0.9848 test accuracy with 80/20, and 0.9792 vs. 0.9794 with 50/50. They differ a little in the first epoch (Figure 1): with 80/20 Adam starts slightly better (training accuracy 0.943 vs. 0.939), with 50/50 SGD does (0.916 vs. 0.906). After three epochs the curves have met. The kind of mistakes differs slightly as well: with 80/20, Adam misses fewer dogs (17 vs. 23) but calls more cats “dog” (20 vs. 15). These are a few images out of 2,500 and well within the margin of error.

The reason is that we only train one linear layer with 1,026 parameters on top of fixed, already very good features. This is essentially a logistic regression: a convex problem without local minima, so both optimizers head for the same solution. Adam’s strength, adapting the step size for every parameter, matters most in deep networks with millions of parameters and has little to add here.

2.4 Metrics

The lecture recommends reporting multiple metrics to avoid measurement bias (Janiesch, 2025), so we report seven. Dog is treated as the positive class. TP are dogs predicted as dog, TN cats predicted as cat, FP cats predicted as dog, and FN dogs predicted as cat.

Metric	Formula	Question it answers
Accuracy	(TP + TN) / all	What share of all images is classified correctly?
Precision	TP / (TP + FP)	When the model says “dog”, how often is it right?
Recall	TP / (TP + FN)	What share of the dogs does the model find?
Specificity	TN / (TN + FP)	What share of the cats does the model find?
F1 score	2 · Precision · Recall / (Precision + Recall)	One number that balances precision and recall
ROC-AUC	area under the ROC curve	How well do the predicted dog probabilities rank dogs above cats, over all thresholds? (1 = perfect, 0.5 = guessing)
Test loss	average cross-entropy	How confident and correct are the predicted probabilities?

For the accuracy we also report a 95 % confidence interval, $\pm\ 1.96 \cdot \sqrt{(p \cdot (1 - p) / n)}$, where p is the accuracy and n the number of test images.

Because both classes are almost equally frequent (for example 1,248 cats and 1,252 dogs in the 80/20 test set), accuracy is a fair summary here. It does not show which class the errors fall on, though; precision, recall and specificity do. ROC-AUC adds a view that does not depend on the 0.5 decision threshold.

2.5 Visualizations (table and graphs)

Table 1 – Test results after 3 epochs

Run	Train / test images	Test accuracy (± 95 % CI)	Precision	Recall	Specificity	F1	ROC-AUC	Test loss
80/20 Adam	9,998 / 2,500	0.9852 ± 0.0047	0.9841	0.9864	0.9840	0.9852	0.9986	0.0458
80/20 SGD	9,998 / 2,500	0.9848 ± 0.0048	0.9879	0.9816	0.9880	0.9848	0.9986	0.0492
50/50 Adam	6,249 / 6,249	0.9792 ± 0.0035	0.9870	0.9712	0.9872	0.9791	0.9986	0.0560
50/50 SGD	6,249 / 6,249	0.9794 ± 0.0035	0.9848	0.9738	0.9849	0.9793	0.9985	0.0564

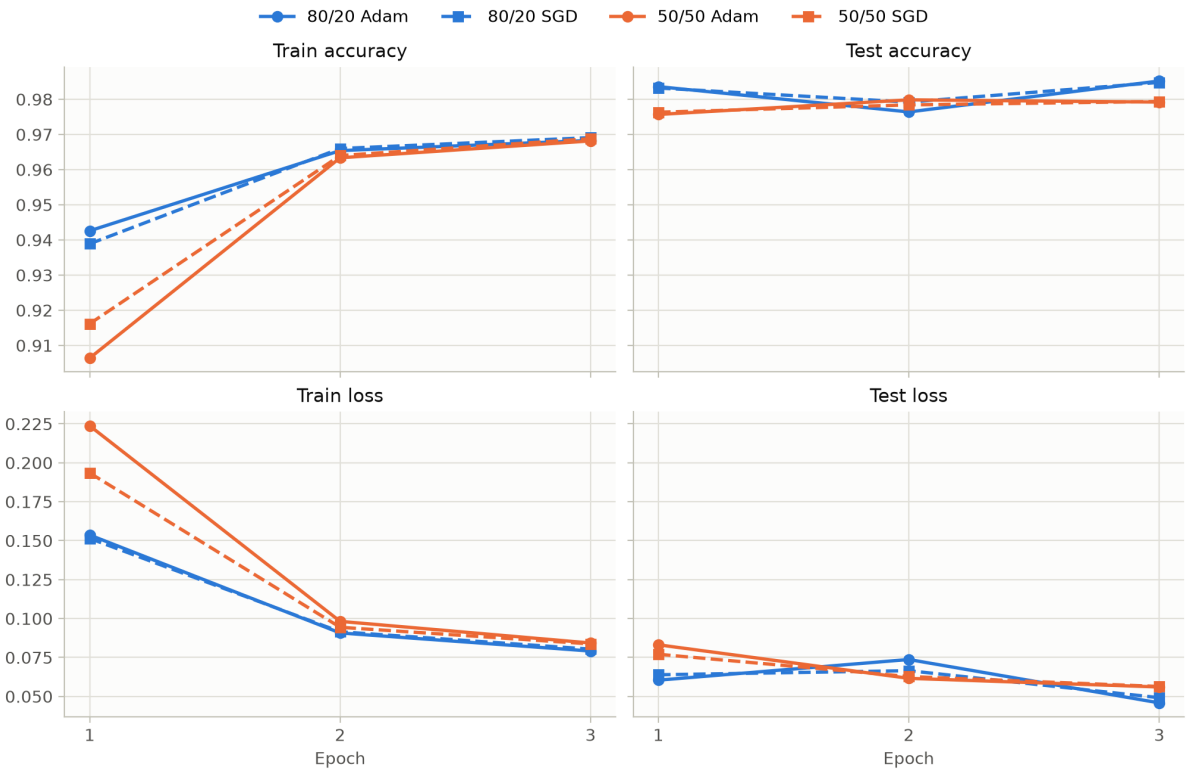


Figure 1 – Accuracy and loss after every epoch. Color shows the split (blue 80/20, orange 50/50), line style the optimizer (solid Adam, dashed SGD). The test accuracy is already about 98 % after the first epoch. The 50/50 runs start lower on the training data but catch up by epoch 2. The small dip of the 80/20 runs in epoch 2 (0.976–0.979) is within the margin of error.

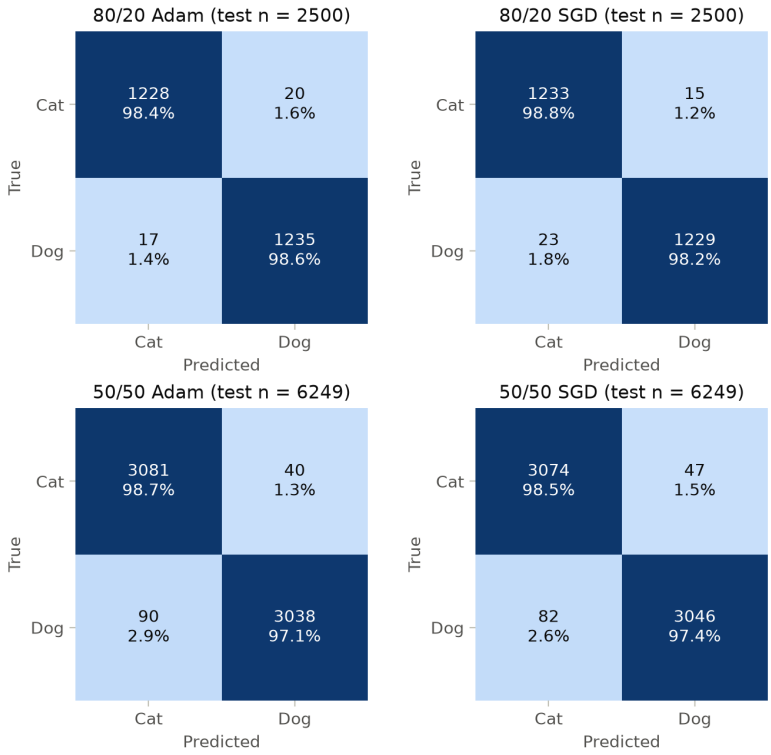


Figure 2 – Confusion matrices. Each cell shows the number of images and the share of its true class. The 50/50 runs lose accuracy mainly by calling dogs “cat” (bottom-left cells).

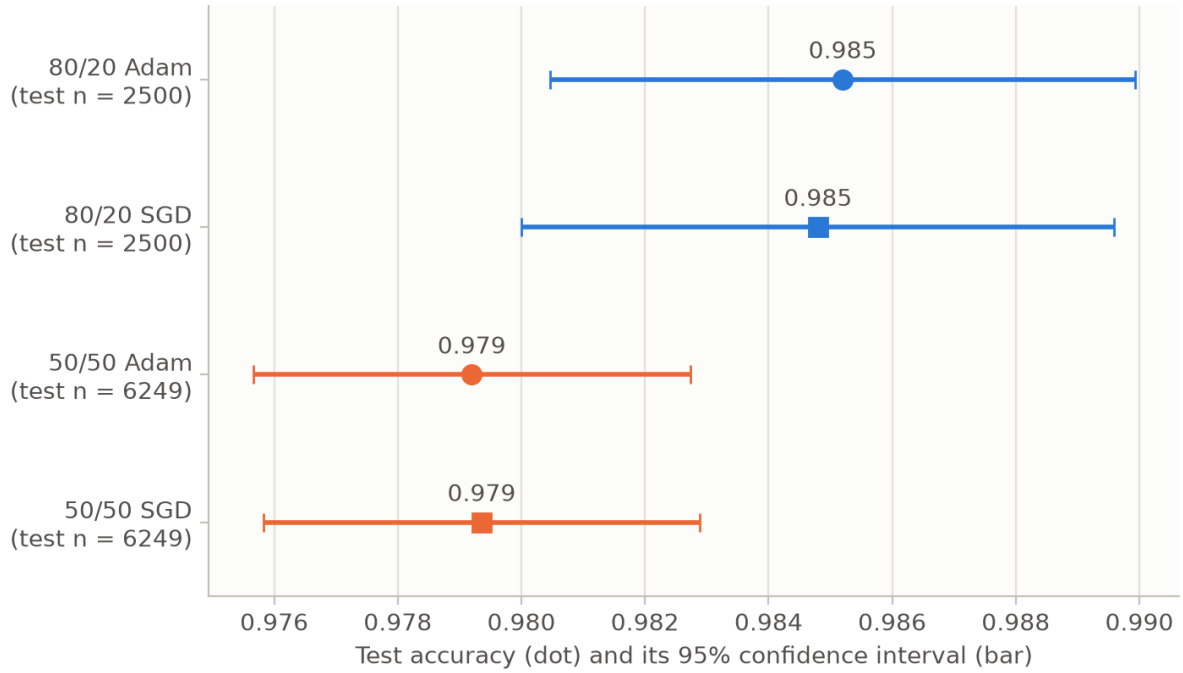


Figure 3 – Test accuracy with 95 % confidence interval. The 80/20 runs are about 0.6 percentage points better, but the intervals overlap slightly. The larger 50/50 test set gives narrower intervals.

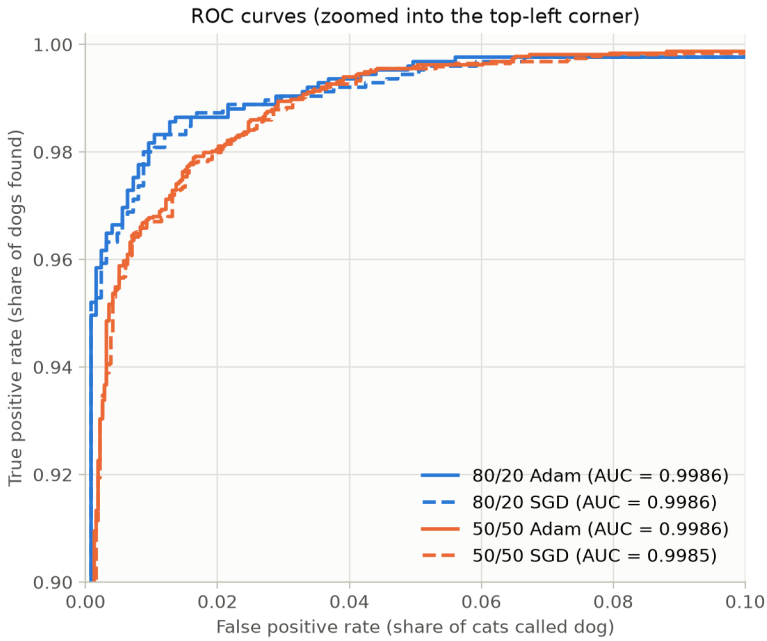


Figure 4 – ROC curves, zoomed into the top-left corner. All four models reach an AUC of about 0.9986. At low false positive rates the 80/20 curves are higher: with a strict threshold they find more dogs while calling few cats “dog”.

2.6 Discussion – why the results differ

- **The split size matters more than the optimizer.** Going from 9,998 to 6,249 training images costs about 0.6 percentage points of accuracy, while switching between Adam and SGD changes almost nothing. With a frozen backbone, the only part that can improve is the last layer, and it benefits from more examples, especially of the more varied dog class.

- **The optimizer hardly matters** because training one linear layer on fixed features is a convex problem with one best solution, and both Adam and SGD with momentum reach it within three epochs.
- **All differences are small.** The best and the worst run are about 0.6 percentage points apart, and every model is far above the 50 % baseline. Each configuration was run only once, with one seed and one split, so differences below about half a percentage point, such as Adam vs. SGD, can be random. The lecture recommends cross-validation to check generalizability (Janiesch, 2025): training and testing on k different splits (for example 5-fold) and averaging the results would show how stable the numbers are.
- **Test accuracy is higher than training accuracy** in every run (for example 0.985 vs. 0.968 for 80/20 Adam). This is not an error. First, the training accuracy is averaged over an epoch while the weights are still changing, whereas the test accuracy is measured once at the end of the epoch. Second, in training mode the BatchNorm layers of the frozen backbone normalize every batch with its own statistics, which adds noise; evaluation uses stable running statistics.
- **No sign of overfitting.** The Case 1 slides read loss curves this way (Ciftci, 2026): a validation loss that falls and then rises again means overfitting. Here the test split plays the role of the validation set. The training loss falls steadily in every run, and the test loss after epoch 3 is lower than after epoch 1 (for example 0.060 → 0.046 for 80/20 Adam); the small rise of the 80/20 runs in epoch 2 does not continue. With only 1,026 trainable parameters, the model has little capacity to memorize individual training images.
- **ROC-AUC is almost identical (0.9985–0.9986)** for all runs, so all models separate cats from dogs equally well when we look at their probabilities. The small differences in accuracy, precision and recall come from where exactly the images fall relative to the 0.5 threshold.
- **The pretrained features do most of the work.** After a single epoch, every model is already at 97.6–98.4 % test accuracy. Further gains would most likely need fine-tuning some backbone layers or data augmentation, not more epochs for the last layer.
- **High accuracy does not explain the model.** The metrics show how often the model is right, but not why. The lecture's wolf-vs-husky example (Janiesch, 2025; Ribeiro et al., 2016) shows a classifier that was right for the wrong reason: it had learned to detect snow, not wolves. Our model could in principle also rely on the background, for example indoor or outdoor scenes, instead of the animal. Explainable AI methods such as LIME, the topic of Case 2, could show which image regions a prediction is based on.

References

Ciftci, S. A. (2026). *Case 1 – Machine Learning und Artificial Intelligence* [Exercise slides]. Advanced Enterprise Computing, Chair of Enterprise Computing, TU Dortmund University.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359–366.

Janiesch, C. (2025). *AI technology foundations* [Lecture slides]. Advanced Enterprise Computing, Chair of Enterprise Computing, TU Dortmund University.

Janiesch, C., Zschech, P., & Heinrich, K. (2021). Machine learning and deep learning. *Electronic Markets*, 31(3), 685–695.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.

Microsoft Cats vs Dogs dataset (Kaggle mirror): https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset

Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill.

Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144.

Smilkov, D., & Carter, S. (n.d.). *A neural network playground*. https://playground.tensorflow.org

A Appendix: Jupyter notebook

Rendered from `notebooks/cats_vs_dogs_classifier.ipynb` in the team repository, with the outputs as saved. The experiment cell (In [ ]) keeps the outputs of the training run that produced `results/*.json`; the other cells were re-run afterwards. Warnings printed by libraries are left out.

Setup

In [1]

```
# If you get import errors, install the packages from requirements.txt (see README.md)

import json, time, random
from pathlib import Path

import numpy as np
import torch, torch.nn as nn
from torch.utils.data import DataLoader, random_split
from torchvision.datasets import ImageFolder
from torchvision.models import resnet18, ResNet18_Weights
from PIL import Image, ImageFile
import torch.nn.functional as F

# use the GPU if there is one (much faster), otherwise the CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

# fixed seed so that runs are reproducible and comparable
SEED = 42
def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

set_seed()

# our helper module for metrics, table and graphs (src/metrics_utils.py)
import sys
sys.path.append("../src")
import metrics_utils as mu
```

```
Device: cuda
```

Load data

In [2]

```
# Use the data folder "1_Case_AEC", which contains a subset of images from:
# https://www.kaggle.com/datasets/shaunthesheep/microsoft-catsvsdogs-dataset
# (Downloading and unzipping might take some time; we require many images to train the
classifier)

# Change this path to where you unzipped the Kaggle dataset so that:
# DATA_DIR/
#   ├─ Cat/
#   └─ Dog/

DATA_DIR = "../data/1_Case_AEC/"  # relative to notebooks/ — every teammate puts the
dataset there


# ImageFolder expects subfolders per class (Cat, Dog)
ImageFile.LOAD_TRUNCATED_IMAGES = True  # tolerate slightly broken JPEGs

# Return True only if `path` is a real, readable image.
```

```
#    What this does:
#    - Tries to open the file with Pillow (PIL) without fully decoding the pixels.
#    - `verify()` performs a quick integrity check (headers, structure).
#    - If the file is empty, corrupted, or not an image, Pillow raises an exception.
#    - We catch that and return False so bad files are filtered out safely.


def ok(path):
    # return True only if the file opens as an image (very short, very robust)
    try:
        Image.open(path).verify()
        return True
    except:
        return False
```

Preprocessing

In [3]

```
# Use the same preprocessing that the pretrained weights expect (easy & reliable)
weights = ResNet18_Weights.IMAGENET1K_V1
preprocess = weights.transforms()

full_ds = ImageFolder(root=DATA_DIR, transform=preprocess, is_valid_file=ok)
print("Valid images:", len(full_ds), "| classes:", full_ds.classes)
```

```
Valid images: 12498 | classes: ['Cat', 'Dog']
```

DataLoader; Train/Test split

In [4]

```
BATCH = 32  # how many images we process at once (fits in memory and speeds things up)
TRAIN_FRAC = 0.8  # 80% of the images for training, 20% for testing

# Split the dataset; the fixed generator gives the same split every time we run the
notebook
n_train = int(TRAIN_FRAC * len(full_ds))
n_test = len(full_ds) - n_train
train_ds, test_ds = random_split(full_ds, [n_train, n_test],
generator=torch.Generator().manual_seed(SEED))

# Training loader: shuffles the data each epoch so the model doesn't see batches in the
same order
train_loader = DataLoader(train_ds, batch_size=BATCH, shuffle=True)

# Test loader: NO shuffling — we just want a stable, honest evaluation
test_loader  = DataLoader(test_ds, batch_size=BATCH, shuffle=False)

# Quick sanity check: how many samples are in each split?
len(train_ds), len(test_ds)
```

```
(9998, 2500)
```

Build a small model (e.g., freeze ResNet-18, train only the last layer)

In [5]

```
# Pretrained backbone => We are building on an existing model with trained weights, this
process is called transfer learning!
model = resnet18(weights=weights)

# Freeze all pretrained layers for speed/simplicity
for p in model.parameters():
    p.requires_grad = False
```

```
# Replace the final layer with a 2-class classifier (Cat/Dog)
model.fc = nn.Linear(model.fc.in_features, 2)
model = model.to(device)

# Cross-entropy is the standard loss for classification (compares predicted class scores
with the true label)
criterion = nn.CrossEntropyLoss()

# Select an optimizer below
# Then, we only pass model.fc.parameters() because we *froze* the pretrained backbone
# and want to train just the final layer. lr=1e-3 is a safe starting learning rate.
# Alternative: torch.optim.SGD(model.fc.parameters(), lr=1e-3, momentum=0.9) - compared in
the experiments below
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Training

In [6]

```
# This might take several minutes due to complex/thorough training and the large number of
images (>12.000)
model.train()  # put the model in *training mode* (enables layers like dropout/batchnorm to
update)

# counters to track total loss and accuracy over the whole epoch
running_loss, total, correct = 0.0, 0, 0
train_true, train_pred = [], []  # labels and predictions of the whole epoch

for imgs, labels in train_loader:          # iterate over mini-batches of training data
    imgs, labels = imgs.to(device), labels.to(device)  # move data to GPU if available
(else CPU)

    optimizer.zero_grad()                  # clear old gradients from the previous step
    logits = model(imgs)                   # forward pass: raw class scores for each image
    loss = criterion(logits, labels)       # compute how far predictions are from true
labels

    loss.backward()                        # backpropagate: compute gradients
    optimizer.step()                       # update trainable parameters using the
gradients

    # accumulate *sum* of losses so we can compute an average later
    running_loss += loss.item() * imgs.size(0)  # loss per sample × batch size

    # turn raw scores into predicted class indices (the highest score wins)
    preds = logits.argmax(1)
    train_true += labels.tolist()
    train_pred += preds.tolist()

    # count how many predictions were correct in this batch
    correct += (preds == labels).sum().item()

    # count how many samples we've processed so far
    total += imgs.size(0)

# compute average loss and accuracy across the entire training set
train_loss = running_loss / total
train_acc  = correct / total

mu.print_metrics(train_true, train_pred, title="Train")

print(f"Train — loss: {train_loss:.3f}, acc: {train_acc:.3f}")
```

```
Train — accuracy: 0.943, precision: 0.940, recall: 0.947, specificity: 0.938, f1: 0.943
Train — loss: 0.154, acc: 0.943
```

Evaluate on the test split

In [7]

```
model.eval()              # put the model in *evaluation mode* (disables dropout, uses
running stats for batchnorm)
total, correct = 0, 0     # counters to compute accuracy across the whole test set
test_true, test_pred, test_prob = [], [], []  # labels, predictions and P(Dog) of the test
split

with torch.no_grad():     # turn off gradient tracking → faster and uses less memory
(we're not training)
    for imgs, labels in test_loader:                # iterate over mini-batches from the
test split
        imgs, labels = imgs.to(device), labels.to(device)  # move data to GPU if available
(else CPU)

        logits = model(imgs)        # forward pass only (no backward): get raw class scores
        preds = logits.argmax(1)    # pick the class index with the highest score for each
image
        test_true += labels.tolist()
        test_pred += preds.tolist()
        test_prob += F.softmax(logits, dim=1)[:, 1].tolist()

        correct += (preds == labels).sum().item()  # count how many predictions were
correct in this batch
        total   += imgs.size(0)                    # update how many test samples we've
seen

# final test accuracy = (# correct predictions) / (total test samples)
test_acc = correct / total
print(f"Test — accuracy: {test_acc:.3f}")

mu.print_metrics(test_true, test_pred, test_prob, title="Test")
```

```
Test — accuracy: 0.984
Test — accuracy: 0.984, precision: 0.985, recall: 0.982, specificity: 0.985, f1: 0.984,
roc_auc: 0.998
```

Experiments: train/test split sizes × optimizers

Each experiment trains a fresh model (frozen ResNet-18, new last layer) with the same seed, so the only differences are the split size and the optimizer. Every run is saved to `results/<run_name>.json` (format described in `PROJECT.md`) so that the metrics, table and graphs can be built from the files.

In [8]

```
def build_model():
    # same setup as above: pretrained backbone frozen, new 2-class last layer
    m = resnet18(weights=weights)
    for p in m.parameters():
        p.requires_grad = False
    m.fc = nn.Linear(m.fc.in_features, 2)
    return m.to(device)


def make_optimizer(name, params, lr):
    if name == "Adam":
        return torch.optim.Adam(params, lr=lr)
    if name == "SGD":
        return torch.optim.SGD(params, lr=lr, momentum=0.9)
    raise ValueError(f"Unknown optimizer: {name}")


def evaluate(model, loader):
```

```
    # returns average loss, accuracy and the per-image labels / predictions / P(Dog)
    model.eval()
    loss_sum, y_true, y_pred, y_prob = 0.0, [], [], []
    with torch.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(device), labels.to(device)
            logits = model(imgs)
            loss_sum += criterion(logits, labels).item() * imgs.size(0)
            y_true += labels.tolist()
            y_pred += logits.argmax(1).tolist()
            y_prob += F.softmax(logits, dim=1)[:, 1].tolist()
    acc = float(np.mean(np.array(y_true) == np.array(y_pred)))
    return loss_sum / len(y_true), acc, y_true, y_pred, y_prob


def run_experiment(train_frac, optimizer_name, epochs=3, lr=1e-3):
    run_name = f"split{round(train_frac * 100)}_{optimizer_name.lower()}"
    print(f"\n=== {run_name} ===")
    set_seed()

    n_tr = int(train_frac * len(full_ds))
    tr_ds, te_ds = random_split(full_ds, [n_tr, len(full_ds) - n_tr],
generator=torch.Generator().manual_seed(SEED))
    tr_loader = DataLoader(tr_ds, batch_size=BATCH, shuffle=True)
    te_loader = DataLoader(te_ds, batch_size=BATCH, shuffle=False)

    model = build_model()
    opt = make_optimizer(optimizer_name, model.fc.parameters(), lr)

    history, start = [], time.time()
    for epoch in range(1, epochs + 1):
        model.train()
        running_loss, total, correct = 0.0, 0, 0
        for imgs, labels in tr_loader:
            imgs, labels = imgs.to(device), labels.to(device)
            opt.zero_grad()
            logits = model(imgs)
            loss = criterion(logits, labels)
            loss.backward()
            opt.step()
            running_loss += loss.item() * imgs.size(0)
            correct += (logits.argmax(1) == labels).sum().item()
            total += imgs.size(0)

        test_loss, test_acc, y_true, y_pred, y_prob = evaluate(model, te_loader)
        history.append({"epoch": epoch, "train_loss": running_loss / total, "train_acc":
correct / total,
                        "test_loss": test_loss, "test_acc": test_acc})
        print(f"epoch {epoch}: train loss {running_loss / total:.3f}, train acc {correct /
total:.3f} | "
              f"test loss {test_loss:.3f}, test acc {test_acc:.3f}")

    result = {
        "run_name": run_name, "train_frac": train_frac, "optimizer": optimizer_name, "lr":
lr,
        "epochs": epochs, "batch_size": BATCH, "seed": SEED, "n_train": len(tr_ds),
"n_test": len(te_ds),
        "train_time_sec": round(time.time() - start, 1), "classes": full_ds.classes,
"history": history,
        "test": {"y_true": y_true, "y_pred": y_pred, "y_prob": y_prob},
    }
    out_dir = Path("../results")
    out_dir.mkdir(exist_ok=True)
    with open(out_dir / f"{run_name}.json", "w") as f:
        json.dump(result, f)
    return result
```

In [ ]

```
SPLITS = [0.8, 0.5]          # 80/20 and 50/50 train/test
OPTIMIZERS = ["Adam", "SGD"]
EPOCHS = 3

results = {}
for frac in SPLITS:
    for opt_name in OPTIMIZERS:
        r = run_experiment(frac, opt_name, epochs=EPOCHS)
        results[r["run_name"]] = r
```

```
=== split80_adam ===
epoch 1: train loss 0.154, train acc 0.943 | test loss 0.060, test acc 0.984
epoch 2: train loss 0.091, train acc 0.965 | test loss 0.074, test acc 0.976
epoch 3: train loss 0.079, train acc 0.968 | test loss 0.046, test acc 0.985

=== split80_sgd ===
epoch 1: train loss 0.151, train acc 0.939 | test loss 0.064, test acc 0.983
epoch 2: train loss 0.091, train acc 0.966 | test loss 0.067, test acc 0.979
epoch 3: train loss 0.080, train acc 0.969 | test loss 0.049, test acc 0.985

=== split50_adam ===
epoch 1: train loss 0.224, train acc 0.906 | test loss 0.083, test acc 0.976
epoch 2: train loss 0.098, train acc 0.963 | test loss 0.062, test acc 0.980
epoch 3: train loss 0.084, train acc 0.968 | test loss 0.056, test acc 0.979

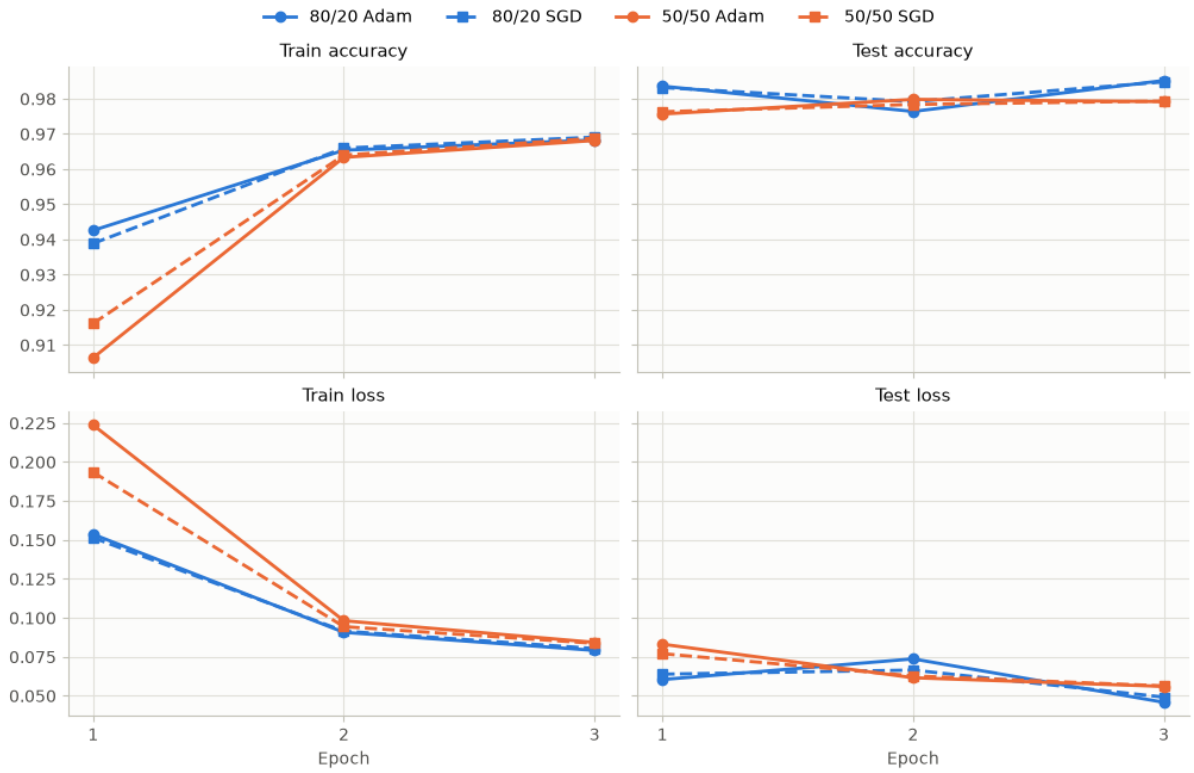
=== split50_sgd ===
epoch 1: train loss 0.193, train acc 0.916 | test loss 0.077, test acc 0.976
epoch 2: train loss 0.094, train acc 0.964 | test loss 0.063, test acc 0.978
epoch 3: train loss 0.084, train acc 0.969 | test loss 0.056, test acc 0.979
```

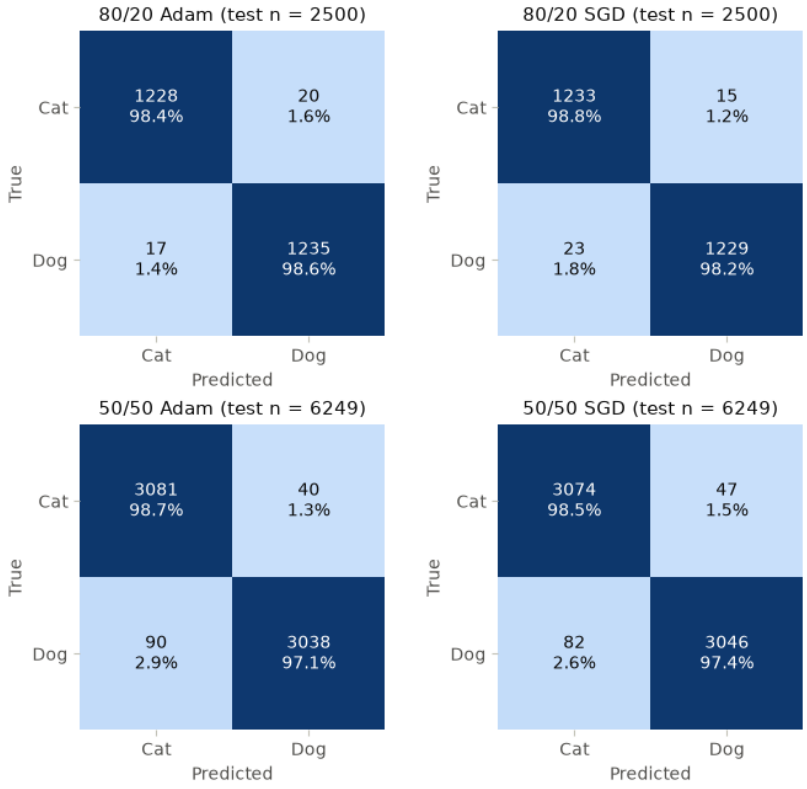
Metrics table and graphs

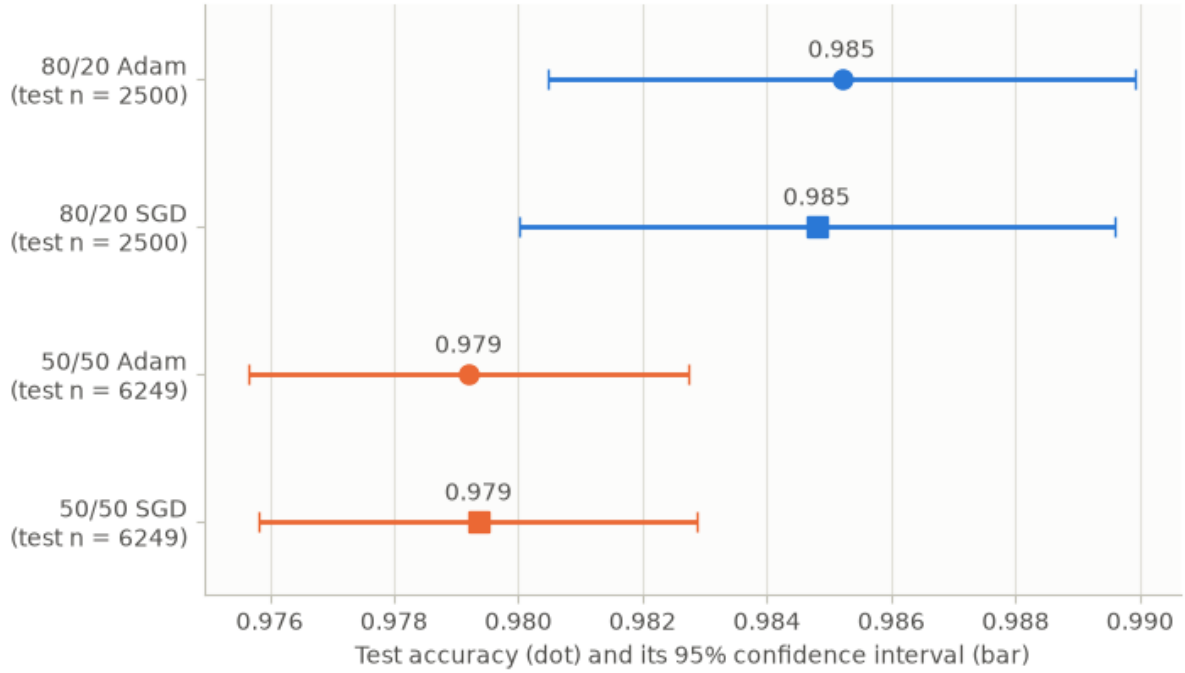
In [9]

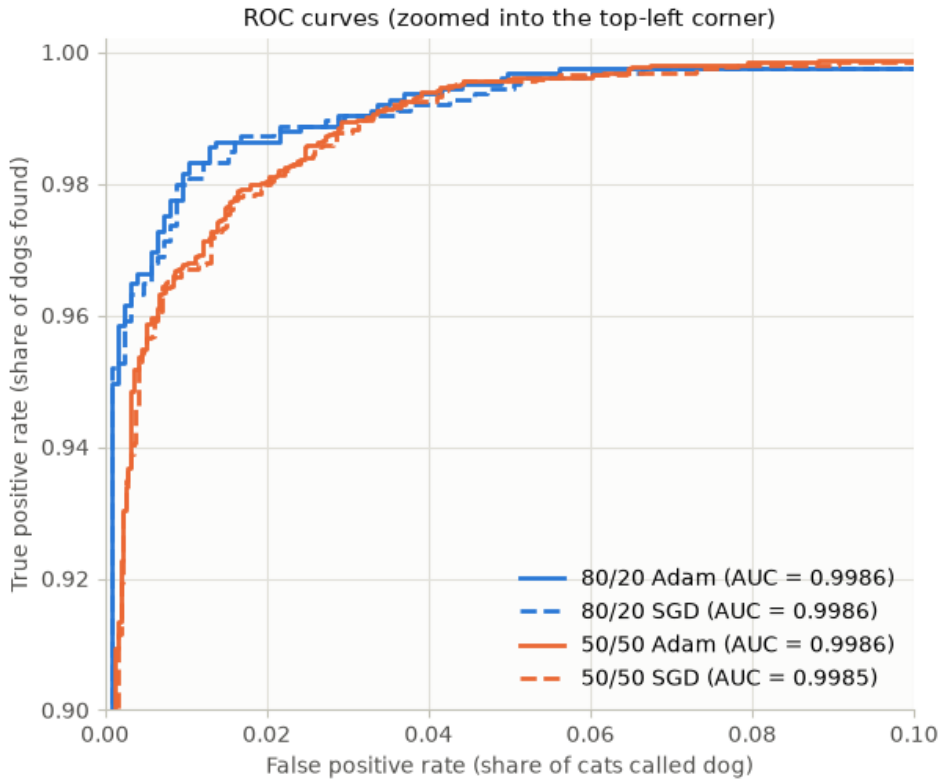
```
# table and graphs for all experiments, computed from results/*.json
runs = mu.load_results()
display(mu.metrics_table(runs))
mu.plot_history(runs)
mu.plot_confusion_matrices(runs)
mu.plot_accuracy_ci(runs)
mu.plot_roc_curves(runs)
```

Run	Train imgs	Test imgs	Train acc	Test acc	± 95% CI	Precision	Recall	Specificity	F1	ROC-AUC	Test loss	FP (cat→dog)	FN (dog→cat)	Time (s)
80/20 Adam	9998	2500	0.9683	0.9852	0.0047	0.9841	0.9864	0.9840	0.9852	0.9986	0.0458	20	17	319.2
80/20 SGD	9998	2500	0.9691	0.9848	0.0048	0.9879	0.9816	0.9880	0.9848	0.9986	0.0492	15	23	323.8
50/50 Adam	6249	6249	0.9682	0.9792	0.0035	0.9870	0.9712	0.9872	0.9791	0.9986	0.0560	40	90	333.4
50/50 SGD	6249	6249	0.9688	0.9794	0.0035	0.9848	0.9738	0.9849	0.9793	0.9985	0.0564	47	82	318.5


80/20 Adam
80/20 SGD
50/50 Adam
50/50 SGD
Train accuracy
Test accuracy
Train loss
Test loss
Epoch
Epoch


80/20 Adam (test n = 2500)
1228
98.4%
20
1.6%
17
1.4%
1235
98.6%
80/20 SGD (test n = 2500)
1233
98.8%
15
1.2%
23
1.8%
1229
98.2%
50/50 Adam (test n = 6249)
3081
98.7%
40
1.3%
90
2.9%
3038
97.1%
50/50 SGD (test n = 6249)
3074
98.5%
47
1.5%
82
2.6%
3046
97.4%
True
Predicted
Cat
Dog





Test the model for single images

In [10]

```
class_names = getattr(full_ds, "classes", ["Cat", "Dog"])

def predict_image(img_path):
    """
    Load ONE image, apply the same preprocessing as training,
    run the model, and print Cat/Dog with confidence.
```

```
    """
    # 1) Open image and ensure 3-channel RGB
    img = Image.open(img_path).convert("RGB")

    # 2) Preprocess exactly like during training
    x = preprocess(img).unsqueeze(0).to(device)  # shape [1,3,H,W]

    # 3) Inference (no gradients)
    model.eval()
    with torch.no_grad():
        logits = model(x)
        probs = F.softmax(logits, dim=1)
        conf, pred_idx = probs.max(dim=1)

    # 4) Human-readable output
    pred_label = class_names[int(pred_idx)]
    print(f"Prediction: {pred_label}  (confidence: {conf.item():.1%})")
```

In [11]

```
# Example use (end the path below with the folder and image you prefer):
predict_image(DATA_DIR + "Dog/995.jpg")
```

```
Prediction: Dog  (confidence: 99.9%)
```